INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KALYANI

Department of Computer Science and Engineering

Software Engineering Project Assignment

MediFlow

Hospital Management System

Assignment 1 · Hospital Management System

Software Test Plan

GROUP 11

Sl.	Name of the member	Registration number
1	Akash Chakraborty	CSE/23011/1059
2	Anipriyo Mandal	CSE/23017/1065
3	Arpan Mandal	CSE/23027/1075
4	Himanshu Patel	CSE/23052/1100
5	Pranjali Sirari	CSE/23072/1120

Live demo at mediflow-hms-6vw1.onrender.com

Source code at github.com/anipriyo/mediflow-hms

Submitted through Google Classroom · October 2026

Group Details

Table 1. Group and assignment details

Field	Value
Group number	11
Assignment number and title	Assignment 1, Hospital Management System
Document	Software Test Plan (document 4 of 5)
Project name	MediFlow Hospital Management System
Problem statement (brief)	Develop a system to manage patients, doctors, appointments, prescriptions, billing and medical records.
Companion documents	Software Requirements Specification (SRS), Software Design Document (SDD), UML Diagrams, Software Test Report
Live demo	mediflow-hms-6vw1.onrender.com
Source code	github.com/anipriyo/mediflow-hms

Table 2. Group members (names and registration numbers)

Sl.	Name	Registration number
1	Akash Chakraborty	CSE/23011/1059
2	Anipriyo Mandal	CSE/23017/1065
3	Arpan Mandal	CSE/23027/1075
4	Himanshu Patel	CSE/23052/1100
5	Pranjali Sirari	CSE/23072/1120

Document control

Table 3. Revision history

Version	Date	Description
0.1	15-Oct-2026	Project selected (Assignment 1, Hospital Management System)
0.2	18-Oct-2026	Scope, user roles and feature set agreed (core, hospital operations, distinctive features)
0.5	21-Oct-2026	Requirements baselined (50 FR, 10 NFR, 12 BR), architecture and UML diagrams
0.9	21-Oct-2026	Prototype implemented, unit, integration, system and acceptance tests executed
0.95	22-Oct-2026	Robustness / edge-case testing, additional defects fixed and regression-tested
1.0	22-Oct-2026	Final report for submission
1.1	22-Oct-2026	Prototype deployed online (Render), live demo link added
1.2	25-Oct-2026	Issued as a separate document (Software Test Plan), one of five

Summary

MediFlow is a web-based Hospital Management System that manages **patients, doctors, appointments, prescriptions, billing and medical records**, extended with laboratory, pharmacy and in-patient (bed and admission) management. Beyond the core requirements, it introduces a **smart queue** with tokens and waiting-time estimates, a **live bed board** with a discharge workflow tied to billing clearance, and **role-based access control with an append-only audit trail** for six staff roles.

This document plans how MediFlow is tested. It sets the testing objectives, scope and strategy, the test design techniques, environment, entry and exit criteria, defect management, roles, schedule and risks, and the approach for unit, integration, system and acceptance testing.

> **Live demo.** The prototype runs online at mediflow-hms-6vw1.onrender.com and opens in any browser with no installation. Sign in with one of the demo accounts (admin, reception, dr.sharma, lab, pharmacy and billing, all with the password Mediflow@123). It uses a free cloud plan, so the first visit after 15 idle minutes takes about a minute to load, and the demo data is reset to fresh records dated that day whenever the server restarts.
>
> **Source code and running it locally.** The full source code is at github.com/anipriyo/mediflow-hms. Install Node.js 20 or later, run `npm install` and then `npm run dev` in the project folder, and open localhost port 5173 in a browser.

Table of Contents

Group Details..... 2
Document control..... 2
Summary..... 3
Table of Contents..... 4
SOFTWARE TEST PLAN..... 5
1. Testing Objectives..... 5
2. Testing Scope..... 5
2.1 In scope..... 5
2.2 Out of scope..... 5
3. Testing Strategy..... 5
3.1 Test design techniques..... 6
3.2 Test environment..... 6
3.3 Entry and exit criteria..... 7
3.4 Defect management..... 7
3.5 Roles, schedule and deliverables..... 7
3.6 Risks and mitigations..... 7
4. Unit Testing..... 8
5. Integration Testing..... 8
6. System Testing..... 8
7. Acceptance Testing..... 9

SOFTWARE TEST PLAN

> This plan follows the structure of IEEE 829 / ISO/IEC/IEEE 29119-3. Test cases are automated and carry the identifier of the requirement they verify, so the plan, the test report and the traceability matrix are generated from the same source.

1. Testing Objectives

- Verify that each of the 50 functional requirements (FR-01 to FR-50) and the measurable non-functional requirements (NFR-01 to NFR-08) are correctly implemented.
- Verify every business rule (BR-01 to BR-12), especially the boundary conditions (slot limits, 5th failed login, 50% discount, 30-day expiry window, payment equal to balance, minimum one bed-day).
- Verify that invalid input and invalid state transitions are rejected with clear messages and without corrupting data (negative testing).
- Verify role-based access control. Every role can perform its tasks and is denied everything else.
- Validate, from the users' point of view, the complete outpatient and inpatient journeys (acceptance).
- Find, record, fix and re-test defects, and provide objective evidence (actual results) for the evaluation of the system.

2. Testing Scope

2.1 In scope

- All thirteen modules listed in the Software Design Document, Section 2, through the REST API and through the user interface.
- Business rules and state machines (appointment, bed, admission, lab order, invoice).
- Security. Authentication, lock-out, token tampering/expiry, logout revocation, RBAC, SQL-injection resistance, password storage, audit integrity.
- Performance of typical API requests on the demo dataset (NFR-01) and responsive layout at phone width (NFR-05).
- Data integrity of multi-step operations (transactions) and printing views (invoice, receipt).

2.2 Out of scope

- Load and soak testing at production scale (NFR-09, NFR-10), requires production hardware.
- Automated cross-browser testing on Firefox and Safari (only Chromium is automated, other browsers are a release-candidate activity).
- Penetration testing by an independent party, accessibility audit (WCAG) and usability testing with real hospital staff.
- Third-party integrations, which are out of the product scope (payment gateway, lab analysers, SMS).

3. Testing Strategy

Testing follows the **V-model**. Each development artefact has a matching test level, and higher levels are executed only when the lower level passes. All tests are automated so that the whole suite can be re-run after every change (regression testing).

Table 4. Test levels (V-model)

Level	Verifies	Scope and approach	Tool	Cases	Exit criterion
Unit	Detailed design (rules)	Pure domain-rule functions in server/domain (slots, conflicts, queue ETA, state machines, billing, inventory, patient validation, lock-out, permissions)	Vitest 3	50	100% pass
Integration	Architecture & module interfaces	REST API → middleware → services → SQLite (in-memory schema, frozen clock) for every module, NFR security/performance checks and robustness probes with invalid, boundary and malicious input (IT-EDGE-*)	Vitest + Supertest	92	100% pass
System	SRS (FR + NFR)	Complete application in a real browser. Built SPA + API + fresh demo database, one test per module workflow, plus a crawl of every screen for all six roles at desktop and phone width (ST-UI-*)	Playwright (Chromium)	14	100% pass
Acceptance	User requirements	End-to-end business scenarios across several roles, validating the user requirements (UR-01 … UR-14)	Playwright (Chromium)	5	100% pass

3.1 Test design techniques

Table 5. Test design techniques

Technique	Where it is applied (examples)
Equivalence partitioning	Valid / invalid mobile numbers, genders, blood groups, payment methods (UT-PAT-02, UT-BIL-05).
Boundary value analysis	Lab reference limits 69/70/100/145 (UT-LAB-02). Reorder level 21/20/0 (UT-PHA-01), expiry at 30/31 days (UT-PHA-02). Discount 50% vs 60% (UT-BIL-02, IT-BIL-02), payment equal to balance vs +₹0.01 (UT-BIL-04), 4th vs 5th failed login (UT-AUTH-01), 1 h, 24 h, 25 h, 48 h stays (UT-IPD-03).
State-transition testing	Valid and invalid transitions of appointment, bed, admission, lab order and invoice (UT-APT-10, UT-IPD-01/02/04, UT-LAB-01, IT-LAB-02, IT-IPD-05).
Decision tables	Discharge clearance. Uninvoiced charges × outstanding balance × role × override reason (UT-IPD-07, IT-IPD-07/08/09).
Use-case / scenario testing	Booking, queue, consultation, dispensing, admission-to-discharge, billing (ST-*, AT-*).
Error guessing & security	Forged/expired tokens, SQL injection strings, double submission, malformed JSON (IT-AUTH-05, IT-NFR-01, IT-NFR-04, IT-PHA-05).

3.2 Test environment

Table 6. Test environment

Item	Configuration
Hardware	Linux x86-64 container, 2 vCPU, 7 GB RAM (comparable to a mid-range laptop)
Software	Node.js v24.21.0, SQLite 3, Chromium 141 (Playwright), Vitest 3.2, Supertest 7
Unit / integration data	Fresh in-memory SQLite database per test file loaded with reference data (8 doctors, 12 lab tests, 26 medicines, 5 wards / 42 beds, 6 patients), clock frozen at Wed 21-Oct-2026 10.00 IST for deterministic results. Bcrypt cost lowered to 4 for speed
System / acceptance data	Fresh demo database generated at server start. 64 patients, ~600 appointments over 14 days, ~380 invoices, 20 active admissions, real wall-clock time
Performance data	Demo database (IT-NFR-03) to 200 mixed API requests

3.3 Entry and exit criteria

- **Entry.** Requirements baselined with IDs. Code builds without errors, database schema creates successfully, test data generator runs.
- **Exit.** 100% of planned test cases executed. All test cases pass, no open defect of severity Critical or High. Every FR covered by at least one passing test. Domain-rule line coverage ≥ 80%.
- **Suspension.** The build fails or more than 20% of a suite fails because of an environment problem. **Resumption.** The cause is fixed and the full suite is re-run.

3.4 Defect management

Every failure is analysed first. If the test script or test data was wrong, the test is corrected and the incident is recorded in the run log (it is **not** a product defect). Product defects are logged with ID, severity, priority, steps to reproduce, expected and actual behaviour, root cause and resolution, and follow the life cycle **New → Assigned → Fixed → Re-tested → Closed** (or **Deferred** with justification). Each fixed defect is protected by a regression test referenced in the defect log.

Table 7. Defect severity levels (priority P1 = fix immediately … P4 = next release)

Severity	Definition
Critical	System unusable, data loss or security breach. No workaround.
High	A main function produces wrong results (e.g. wrong bill) or is blocked. Workaround is difficult.
Medium	Function works with a wrong or confusing detail. Workaround exists.
Low	Cosmetic, wording or minor usability issue.

3.5 Roles, schedule and deliverables

Table 8. Test activities and responsibilities

Activity	Responsibility	Output
Test planning and design	Test lead with requirements owner	This test plan, test case catalogue embedded in the test code
Unit & integration test implementation	Developers of each module	tests/unit, tests/integration
System & acceptance test implementation	Test engineer acting as end users of each role	tests/e2e, screenshots
Execution and defect logging	Test engineer	Run log, actual results, defect log
Re-testing and regression	Developers + test engineer	Updated run log
Test summary report	Test lead	Software Test Report

3.6 Risks and mitigations

Table 9. Test risks

Risk	Mitigation
Time-dependent rules (slots in the past, queue ETA, bed-days, session expiry) make tests flaky.	Central clock that tests freeze. E2E tests book only for the next day and use seeded same-day appointments.
Shared database between E2E tests causes order dependence.	Each E2E test creates or looks up its own data through the API before acting in the UI. Fresh database for every run.
Tests written by the same team that wrote the code share its blind spots.	Boundary and negative cases derived from the requirements, not from the code, exploratory walkthroughs by members not involved in the module.
Short project schedule.	Full automation. The complete regression suite runs in about two minutes.

4. Unit Testing

Objective. Verify each business rule in isolation. **Approach.** White-box tests of the pure functions in `server/domain`, applying boundary-value and state-transition techniques. No database or network is involved, so each test runs in milliseconds. **Coverage.** Domain-rule modules 99.2% lines / 85.9% branches. Whole server 95.45% lines / 84.15% branches (unit + integration suites, V8 coverage).

Table 10. Unit test coverage by module

Module under test	Rules verified	Cases
appointmentRules, time	Slot generation, working days, slot validation, overlap detection, free slots, appointment transitions	11
queueRules	Priority ordering, token sequence, ETA with and without consultation in progress	5
bedStateMachine, admissionRules	Bed transitions, bed-days, proportional allocation after transfer, discharge clearance	7
billingRules, money	Totals and discount rounding, discount limits, payment validation, invoice status, number formats	8
labRules, inventoryRules	Lab transitions, abnormal flag, low stock, expiry window, dispensability, alerts	7
patientRules, authRules, permissions	ID format, validation, duplicates, age, lock-out, password policy, RBAC matrix	12

5. Integration Testing

Objective. Verify that the API, middleware, services, rule modules and database work together, including transactions, constraints and the audit log. **Approach.** Grey-box tests that send real HTTP requests to the Express application (Supertest) backed by a fresh in-memory SQLite database with the production schema. Users log in through the real login endpoint and act with their own tokens. Integration order is **bottom-up**. Rule modules were unit-tested first, then each service was integrated through its API. Each test checks the HTTP status, the response body and, where relevant, side effects such as charges, stock levels, bed status and audit entries.

Table 11. Integration test areas

Area	Interfaces exercised	Cases
Authentication & users	/auth/*, /users/*, login, lock-out, deactivation, token tampering, logout, RBAC, user admin	10
Patients, doctors, appointments	/patients/*, /doctors/*, /appointments/*	15
Smart queue	/appointments/{id}/check-in, /queue/*	7
Consultation, laboratory, pharmacy	/consultations/*, /lab/*, /pharmacy/* incl. charge creation and atomic dispensing	17
Admissions, beds, billing	/beds/*, /admissions/*, /billing/* incl. clearance and receipts	18
Reports, audit, NFRs	/reports/*, /audit, SQL injection, password storage, headers, latency	9
Robustness / edge cases	Impossible dates, malformed filters, wildcards, paging limits, over-long or wrong-type input, ownership and duplicate checks, sequence-number uniqueness, time-zone day boundaries	16

6. System Testing

Objective. Verify the complete, integrated system against the SRS from the user interface. **Approach.** Black-box tests with Playwright driving a real Chromium browser against the production build served by the Node.js server with a freshly generated demo database. Each test logs in through the login form as the relevant role and performs

a complete module workflow (e.g. book → check in → queue, generate invoice → partial payment → overpayment rejected → full payment → print receipt). Non-functional system checks include role-based menus and access-denied pages (security), responsive layout at 390 px (usability, NFR-05) and API latency (performance, NFR-01, measured in IT-NFR-03).

Table 12. System test cases

ID	System test
ST-APT-01	Book an appointment for tomorrow with the 4-step wizard
ST-AUTH-01	Login per role, role-based menus and blocked direct URLs
ST-BIL-01	Generate an invoice, take a partial UPI payment, reject an overpayment, settle and print the receipt
ST-CON-01	Doctor calls the next patient, records the consultation and completes it
ST-IPD-01	Admit a patient from the bed board and transfer to another bed
ST-LAB-01	Lab technician collects a sample and records an out-of-range result
ST-PAT-01	Register a patient through the form with validation feedback
ST-PHA-01	Pharmacist cannot dispense an out-of-stock prescription but dispenses a valid one
ST-QUE-01	Check in today's appointment and see the token on the queue board
ST-RPT-01	Administrator views dashboard, reports, users, doctors and the audit trail
ST-UI-01	Every screen renders for every role at desktop width without errors
ST-UI-02	All screens fit a 390 px phone without horizontal page scrolling
ST-UI-03	Broken or unauthorised deep links show a clear message instead of crashing
ST-UI-04	Double-clicking “Record payment” records only one payment

7. Acceptance Testing

Objective. Confirm that the system satisfies the user requirements (SRS, Section 7) and is ready for use. **Approach.** Scenario-based acceptance tests that follow a patient through several roles in one continuous flow, written from the acceptance criteria of the user stories. In this academic project the team acted as the hospital users. The scenarios are scripted and automated (Playwright) so that they are repeatable. A real deployment would add a supervised user-acceptance session with hospital staff and formal sign-off.

Table 13. Acceptance test scenarios

ID	Acceptance scenario	User requirements validated
AT-01	OPD patient journey. Check-in → consultation → lab → pharmacy → billing	UR-02, UR-04, UR-05, UR-06, UR-07, UR-10
AT-02	IPD journey. Admission → discharge request → billing clearance → discharge → bed turnaround	UR-08, UR-09, UR-10
AT-03	Each role sees exactly the menus it is entitled to	UR-13
AT-04	Usability. Booking in at most 4 steps and no horizontal scrolling on a 390 px phone	
AT-05	Administrator can trace the journeys in the audit trail	UR-14

Acceptance criteria. All acceptance scenarios pass. No open Critical/High defects, every user requirement UR-01 … UR-14 is validated by at least one acceptance or system test.